

also meets all the conditions described before. It's a .haml file and the matching string is wrapped in a string decorator ().

1. Now that we found that match let's swap the text for Update your project name, topics, description, compliance framework, and avatar. and save the file. If you go back to your running GDK in your browser and you refresh the page, you should see the change reflected in the UI.

1. We have completed a change in the UI and it's time to wrap it up by doing some preparation tasks before opening an MR:

- Run the translation job in your Terminal: This is a process that updates the translation files with your string changes. To do so, simply copy and paste this command in your Terminal: `bin/rake gettext:regenerate`. This will regenerate the `gitlab.pot` file mentioned before. Have in mind that this job can take several minutes and the terminal won't give you immediate feedback so just be patient.

- Add a changelog: The changelog is a brief explanation of what the change is doing. The changelog is generated through an interactive command prompt that can be launched from the Terminal with the following command `bin/changelog 'Your changelog message'`. After entering this command simply follow the Terminal prompts.

1. Finally, we are ready to open an MR with this change against the main GitLab repo. Don't worry about this step. You won't break anything and your MR can be closed later without affecting anything in the main repo. To push your change you want to do two things:

- Enter the following git command in your terminal:
 - `git add ..` This command will add the files that you just changed so they can be committed.
 - `git commit -m 'A meaningful message'`. This command will create a commit with your changes. It's very important that you comply with the commit message rules, otherwise your pipeline will fail. To comply with the rules make sure that your commit message it's a least three words but not more than 75 characters, the first letter needs to be capitalized and it can't end in period ..
 - `git push --set-upstream origin your-branch-name`. This command will create an upstream branch and push your changes against that remote branch. After the push is completed the terminal will show the MR url which you can open in a browser to finalize your change.

1. You're done! Just make sure that you add all the final details to your MR including a milestone and a type label

Caveats

Strings that are tied to a test

The above example shows the process for a text string that doesn't have a test. Although many UI strings are not tied to any particular test (and therefore spec file modifications aren't needed), there are test assertions that check for UI text. Usually what those tests are doing is verifying for the existence of a particular UI component by reading the inner text of an element.

If you change a UI string that is tied to a test and you don't modify the text in the spec

file, the test will fail since it cannot locate the string anymore.

You can find out if the text you're changing is tied to a test if it also shows up in your VS Code search results inside a .spec.js or .spec.rb file.

In general, you only need to swap the old text with the new one in the spec files to have a passing test. However there is a small number of cases where swapping a text may require some extra modification to the spec file but those are rare.

Finding generic strings

The above example shows the process for a text string that is very specific. Very specific strings are easy to find and locate because it's likely that there's only one instance of that text in the whole app. But what about generic strings like the text inside some buttons, for example 'Expand'?

For those cases where you need to replace a generic string, you might have to do some logical thinking to identify the correct file and text instance.

One good strategy is to find a very specific text string near the generic string. If you search for the specific string you will likely find it in the template or component file that contains the other text. Once you have found that file you can perform a search scoped to the file by pressing CMD + F and searching for the generic string inside that file. Most likely any result inside that file is the correct reference to the text you're attempting to change. However, always verify that your change is indeed reflected by checking your live GDK environment.

SECTION: product/ux/ux-resources/experimentation.md

Ever wonder if we do AB Testing, aka Experiments?

Yes, we do experiments in our app! Most experiments are run by the Growth team on the SaaS product, and the majority are done on our registration, trial and purchase flows. We also collaborate with stage groups and run experiments around increasing the visibility, usability and learnability of a feature. These experiments are ultimately aimed at improving KPIs that tie back to revenue. Things like increasing engagement with the product, increasing the speed at which a customer uses a feature or upgrades their account, or even increasing the number of stages used by a namespace.

What is experimentation?

Experimentation is a way to validate and measure improvements to a digital product against particular goals. Basically, we have an idea that if we change something in our product, our users will respond to it, and the metric that we want to impact will be affected by that. Experimentation works well when there's a large number of users and a good baseline product (see the When is an experiment not a good idea? section).

Terms we use at GitLab

Our vocabulary is a bit different when we're talking about experimentation and UX, so here are a few important terms and how we use them here.

- Experiment: We usually mean an A/B or multivariate test when we say this.
- Hypothesis: This is usually a one sentence statement about what we think will change as a result of our experiment. For example: "We believe the middle name field of the sign-up form is disruptive. To verify that we will remove the middle name field from the sign-up form, and we'll measure this by observing an increase in the sign-up rate. We'll call this experiment a success if we observe a $\geq 20\%$ increase in sign-ups."
- Variant/Control: The control is the "current" user experience. The variant is the "proposed design."
- Success metric: This describes the moment a user takes a desired action in the scope of an experiment. For example, out of all the users who were exposed to the variant where a new button is added to the UI, how many clicked on that button.
- Adoption: This is about how a user (or an account) establishes a relationship with our product. It's beyond conversion - it's about how habits are formed. For example, just because someone merges one merge request, it doesn't mean they have adopted the feature. But maybe once a user creates a merge request and adds a team member to review it, they are more likely to continue using the product. That knowledge would inform our adoption metric.
- Expansion: This is about how an account grows. It comes after activation. Once a user is considered activated, what makes them most likely to use more features, level up to a higher tier, or add more users to their subscription?
- Retention: This refers to how many users (or accounts) remain active in a period of time. For example, if 100 new namespaces are created on Day 0, how many are still active after 1 month, 3 months, 6 months?
- AHA Moment: This is a moment (or moments) in the product, where a user gets it! They understand the value of the product, and exactly how it will help them. An example of an AHA moment in GitLab would be when a pipeline successfully runs for the first time.
- Statistical significance: A throwback to statistics class, this is a way of describing how confident we are in results. If we say an experiment was a win and it reached significance, it means we are confident the experiment was successful because of the change we made, and not some other reason. If we say an experiment didn't reach significance, it probably means that the measured impact was either very small, there was no impact at all, or didn't have enough users in the experiment group.
- Time to significance: This is how long it will take to reach significance/confidence in the results. It's a function of how many users will see the control and variant. This can span from a couple of days to a couple of weeks. It depends on how much traffic the area that is being experimented with gets.

The Growth section has groups related to Conversion, Activation, Expansion and Adoption. The things these groups work on relate back to the definitions above, but they aren't exactly the same.

When and why should experimentation be used?

Use an experiment when you need to quantify the impact of product changes (eg. click rate, conversion rates, other metrics) or when you need to validate a solution.

When is an experiment not a good idea?

There are some situations where we'd prefer to make a change to the product and roll it straight out to product. These are:

- when we are very confident that the change is the right one, and will result in a big usability improvement. We would do this so that all users can immediately benefit from the improvement. These changes can be small or large. They can vary from adding a label to an icon-only button to fixing a flow that spans across a few pages.
- for a 'big bet'. This is a bigger change, such as a redesign of the sign-up flow, where we are aligning to overall best practices, we're informed by data and we've conducted other forms of solution validation like user testing and other UX research methods.
- it seems pretty obvious, but we wouldn't use an experiment to decide how to fix a bug.
- when it will take a long time to reach significance and the impact to revenue is uncertain/low and/or the impact to users is very high/certain. In this case we can make the change, or reject the experiment. Growth groups use the ICE framework for prioritizing experiments.
- when the baseline experience is poor, below a B- using our heuristic scoring. In this case, we would require initial improvements to the experience before proceeding to an experiment.

How do we design for an experiment?

Much of the product design process for Growth will remain familiar, but there are some differences. There are also considerations and concepts we need to understand and follow.

- For an experiment, we work from an experiment write-up (add link) instead of user stories and/or JTBD. But we still need to be familiar with the JTBD of the feature we're experimenting with.
- We always want to have a well-written experiment before we start designing. The minimum required should be the hypothesis, the business problem, whether there's any supporting data, and what is the expected outcome. To help you get started, use the Experiment Idea issue template.
- A north star metric, combined with the vision of the user experience is what should be driving most, if not all experiments. To get there, a designer should establish a UX strategy in collaboration with their product manager. Examples of such UX strategies can be seen [here](#) and [here](#).

Do experiments replace user research?

No, they don't. This is because experiments tell us what is happening while user research can tell us why something is happening. We do both problem and solution validation research in Growth. Here are examples of things we might ask:

- For problem validation, we might want to understand what users are looking to learn during a trial. Insights from this kind of research could inform both UX improvements and experiment ideas.
- For solution validation, we might run a usability study on a big bet. We also run usability studies on key flows every quarter or so, in order to get qualitative feedback on our launched experiments.
- Sometimes experiments are inconclusive, or surprising because they don't match what we expected. When this happens, we'll conduct user research to find out the 'Why'.

Crafting a good experiment starts with a good hypothesis

- A good hypothesis is a statement that can easily be validated as true or false.
 - Template for writing a hypothesis statement: We believe that insert the change you are going to test will result in insert desired result. We'll know this is true when we see a insert target change in metric.
 - An example of a good hypothesis statement: We believe that adding a checkbox to enable SAST scanning to the project creation page will result in higher user adoption of the Secure stage. We'll know this is true when we see a 5% increase in Secure adoption.
- A hypothesis tests something that matters. If users won't care, or the company doesn't care, then it's not a great thing to test.
- The hypothesis should test one thing. Avoid things like "we think that X and Y are true" or "we think that X will result in X and Y".
- If you expect an experiment will have an impact on multiple metrics make sure you list all of them ahead in the experiment write-up.
If you want to impact trailing metrics in addition to an initial metric, make that very clear in your experiment write-up.
- After you collect data, if you've written a good hypothesis, you should be able to restate it as your conclusion. For example:
 - Hypothesis: We think that adding a koala image to the page will increase clicks by 10%.
 - Conclusion: Adding a koala image to the page resulted in clicks increasing by 12.4%, at a 97% significance.

Picking the right metrics

TBD

Things to watch out for

Experimentation is an awesome way to improve your product. There are a lot of considerations though, so think through your goals and constraints to ensure it's the right approach.

- Launching experiments may cause delays compared to just introducing a change. These delays may be caused by additional engineering work in order to launch experiments, longer times to reach statistical significance, lack of experience with them in teams outside of Growth.
- If your team hasn't experimented yet, ask for guidance from Growth. Reaching out to product designers should be your first step. They'll be able to help, or connect you with people that can help with data analysis, technical challenges of launching experiments, avoiding result misinterpretation, and make sure that your experiments don't test too many things at once.
- If your team has some experience with experimentation, you should still check in with Growth and check whether there are any conflicting experiments currently running.
- Consider how much traffic the area that you want to experiment with gets. Is it low? Avoid doing an experiment altogether. Is it moderate? Acknowledge that it may take time to reach a statistically significant result.

Resources

- Experiment Guide

Have an idea for an experiment?

Get in touch with one of the product designers on Growth or open an issue with the dedicated template.

SECTION: product/ux/ux-resources/designers-guide-to-contributing-ui-changes-in-gitlab/index.md

Contributing to GitLab's main codebase

GitLab's main codebase contains all the code for the GitLab project including its UI. Technical knowledge is required to contribute, but it's not necessary to have a deep understanding of programming concepts to do meaningful contributions. There are multiple opportunities to improve GitLab by doing small UI text changes or small CSS refactorings.

Contributing to GitLab's code requires HTML, CSS, Terminal (CLI), and Git Knowledge. Basic Ruby and JavaScript knowledge is also encouraged.

Anyone contributing to GitLab's main codebase should have a working and running GDK, be familiar with using a code editor (preferably Visual Studio Code) and know the basic Git commands.

As a general rule, if it can be fixed in a browser inspector, it can be probably be fixed it for real in the GitLab codebase.

Everyone is encouraged to find small UI issues and submit their changes via a merge request (MRs).

GitLab's main codebase contribution basics

Step 1: Install the GDK

The GitLab Development Kit (GDK) provides a local GitLab instance that allows you to test changes locally, on your workstation.

1. Install the GDK on your machine
1. Learn how to run the GDK
1. GDK Commands cheatsheet

Step 2: Understand GitLab's codebase structure

Three kinds of files handle UI rendering. At its core, GitLab is a Ruby on Rails application. The Ruby on Rails application renders GitLab's front end with .haml files. HAML (HTML Abstraction Markup Language) is a Ruby-based HTML template system. It's easy to learn and it even closes HTML tags for you!

For Stylesheets, GitLab uses a CSS pre-processor called SASS. SASS (Syntactically Awesome Style Sheets) uses .scss files that handle all of the usual stuff CSS does, but with a bit more sophistication that helps us keep GitLab's CSS better organized.

Finally, for interactivity and client-side application logic, GitLab uses a framework called Vue.js. It's rare to have to change the .vue files unless you're changing a Pajamas component, or creating a new one.

Step 3: Choose a code editor and start making small changes

You can pick any code editor you like, but Visual Studio Code is a safe bet and the industry standard.

Once you have your code editor installed, open the gitlab-development-kit/gitlab folder. It contains all of the GitLab files that change when new features and contributions are added.

A good first step is to find some text inside the product to update. Use the search feature in your code editor to locate where the text exists in the code base, change the text to something else, and save the file. When you reload your local instance of GitLab, you should see that change reflected in the UI.

Step by step guide to contributing UI code changes

For a detailed step by step guide of how to make UI code changes against the GitLab's main codebase visit this page:

Step by Step Guide to Make Code Changes

Finding opportunities to do UI code contributions

{{< youtube "GUTalaJtqU" >}}

How to Make UI text changes in GitLab (Designer Edition) by @jj-ramirez

How to Make UI text changes in GitLab (Designer Edition) by @jj-ramirez

{{< youtube "AEv3XFW0xJQ" >}}

This page is adapted from a beautifully designed PDF created by @jj-ramirez ·

SECTION: product-development/_index.md

This section of the handbook is for content that is jointly shared by Product and Engineering. For any content wholly owned by only one division, please use the relevant sections: Product or Engineering.

SECTION: product-development/product-development-flow/_index.md

Please follow this guidance on how to contribute to this page. <!-- To see a list of the latest updates to this page, visit Product development flow releases. -->

Overview & philosophy

GitLab's product mission is to consistently create products and experiences that users love and value. To deliver on this mission, it's important to have a clearly defined and repeatable flow for

turning an idea into something that offers customer value. Note that it's also important to allow

open source contributions at any point in the process from the wider GitLab community - these won't necessarily follow this process.

This page is an evolving description of how we expect our cross-functional development teams to work, and reflects the current workflow being leveraged. All required actions or outcomes in this page are denoted as follows:

> *Denotes a required aspect of the product development flow.*

Feature development is expected to pass through all phases to achieve specified outcomes, while the rest of the workflow should be considered as a set of best practices, tools, and recommendations. We realize there are unique cases in which certain product improvements may not need to flow through all the phases. We trust product managers to use their best judgement with alignment from their design and engineering team. The goal of this page is to support teams in their workflows by highlighting the necessary outcomes to target in each phase as well as sharing strategies/tactics, activities, teams can employ to achieve these outcomes.

Additionally, this page aims to clarify the minimal set of required actions, such as labels, needed across all phases to keep the product system efficient in terms of tracking, searching and cross-functional collaboration. To maintain clarity and avoid confusion, we do not list optional actions on this page but teams may choose to employ additional actions, such as labels for planning, even if they are not mentioned on this page.

As teams leverage the product development flow, they may find that certain strategies/tactics are serving their teams well toward success. Therefore, we welcome MRs to this page, so we can create a robust playbook of options to build valuable features for customers. All team members are encouraged to follow the change process for this page to share their best practices.

Measuring Success

We take both a qualitative and quantitative approach to measuring success of the Product Development Flow. Details on the measures and results are available on this handbook page.

But Wait, Isn't This Waterfall?

No. Although the phases described on this page appear to be independent and linear, they are not. They're presented in this way for simplicity and ease of navigation. At GitLab, we do not

promote working in a linear manner. Phases in the product development lifecycle may overlap or occur in parallel.

We aim to achieve key outcomes in each phase in order to de-risk subsequent phases. However, the product development flow doesn't dictate the order we go through the phases, or the time spent in each. When teams have a high confidence in their direction, they should feel empowered to skip or shorten phases that won't contribute to improved confidence.

Examples:

- An engineering team conducts a technical review while other team members are performing Validation Phase activities. The team can then move to the Build phase rapidly with high confidence that their improvement is good for customers and technically feasible.
- A bug is reported by a GitLab customer. The Product Manager tests the bug and confirms its existence (Problem Validation). The team is extremely confident in the solution, so Design and Solution Validation are not needed. The bug is moved immediately to Build.

Workflow Summary

<object data="/images/product-development-flow/PDF-Diagram.svg" style="width: 100%;" type="image/svg+xml">

Product Development Flow diagram.

Unable to load this content, check console for details.

</object>

> <i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> We use workflow labels to efficiently communicate an issue's state. Using these labels enables collaboration across teams and communicates an issue's current state.

The workflow labels are prefixed with workflow::, such as workflow::ready for development. The following diagram shows how a new issue moves to each workflow label, though states can be skipped when appropriate. The rest of this document describes each workflow step in detail.

mermaid

stateDiagram-v2

[] --> ValidationBacklog: New issue created

ValidationBacklog: validation backlog

ProblemValidation: problem validation

ReadyForDesign: ready for design

Design: design

SolutionValidation: solution validation

ValidationBacklog --> ProblemValidation

ProblemValidation --> ReadyForDesign: (Optional) Needs design

ReadyForDesign --> Design

ProblemValidation --> SolutionValidation

Design --> SolutionValidation

PlanningBreakdown: planning breakdown
Scheduling: scheduling
Refinement: refinement
ReadyForDev: ready for development
InDev: in dev
InReview: in review
Verification: verification
Complete: complete
Blocked: blocked
Security: awaiting security release

SolutionValidation --> PlanningBreakdown: PM signals intent to prioritize
PlanningBreakdown --> Scheduling: (Optional) Awaiting scheduling
PlanningBreakdown --> Refinement: (Optional) Needs refinement
PlanningBreakdown --> ReadyForDev: Has type label & weight. Prioritization requires milestone, and Deliverable label.
Scheduling --> ReadyForDev: Milestone assigned
Refinement --> ReadyForDev
ReadyForDev --> InDev
InDev --> InReview: MRs ready for review
InReview --> Verification: MRs merged
Verification --> Complete: Verified in staging/prod

InDev --> Blocked: Blocked by dependency/question
Blocked --> InDev: Block resolved

InReview --> Security: Waiting for monthly security release
Security --> Verification: MRs merged and released in security update

Complete --> []

Issue descriptions as the Single Source of Truth (SSOT)

> <i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Issue descriptions shall always be maintained as the single source of truth.

It's not efficient for contributors to need to read every comment in an issue to understand the current state.

- Issue description accuracy should be maintained by the DRIs throughout each phase. However all collaborators can and should contribute when they see discrepancies or needed updates.

Validation track

For situations when the customer problem isn't well understood, Product Managers (PMs) and the User Experience Department (UXers) should work together to validate new opportunities before moving to the Build track. The Validation track is an independent track from the always moving Build track. PMs and UXers should work together to get at least two months ahead, so that the Build track always has well-validated product opportunities ready to start. Milestone work

should be prioritized with the understanding that some milestones may include more validation efforts than others. Validation cycles may not be necessary for things like bug fixes, well understood iterative improvements, minor design fixes, and technical debt.

Validation Spectrum

The types of activities and depth of research required in the validation track will depend on how well we understand the customer problem and solution.

Validation Goals & Outcomes

When: When our confidence about the proposed problem or solution isn't high. For example, if we aren't reasonably sure that the problem is important to a significant number of users, or that the solution is easy to understand and use.

Who: Product Manager, Product Designer, UX Researcher, Product Design Manager, Engineering Manager

What:

- Understand the user problem we are trying to solve.
- Identify business goals and key metrics to determine success.
- Generate hypotheses and research/experiment/user-test.
- Define MVC and potential future iterations.
- Minimize risks to value, usability, feasibility, and business viability with qualitative and quantitative analysis.

Outcome: We have confidence that a proposed solution will positively impact one or more Product KPIs. There may be reason for exceptions, so the team would need to be clear in that case, and be able to justify that it's still important without mapping back to our KPIs.

If we don't have confidence in the MVC or what success looks like, we should continue validation cycles before we move to the Build track.

Validation phase 1: Validation backlog

<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Label: workflow::validation backlog

Key Participants

Role	Function
DR	Product Manager

| Collaborators | Product Designer
 Product Design Manager
 Engineering Manager |
| Informed | Customers
 Customer Success Manager
 Product Marketing Manager

 Other stakeholders as appropriate |

Description

The growth of a world class product is built from a well maintained backlog. Product Managers are responsible for refining a group's backlog to ensure validation opportunities are scoped and prioritized in line with category direction, stage, and/or section level strategy. The backlog is also the single source of truth for stakeholders to understand and engage with your group. An issue position in the backlog, along with the description, discussion, and metadata on those issues are key pieces of data necessary to keep stakeholders up to date.

Outcomes and Activities

| Outcomes | Activities | DRI |

|-----|-----|-----|

| <i aria-hidden="true" style="color:rgb(252,109,38); font-size:1.25em" class="fab fa-gitlab fa-fw"></i>Up to date issues and epics: At GitLab, issues are the single source of truth for any change to the product. Keeping these up to date increases efficiency and transparency by allowing all team members to understand the planned work. | - Create issues in response to a sensing mechanism. Consider using the Problem Validation issue template for new features.
- Review issue discussions and update relevant info in the description.
- Keep metadata (such as labels) up-to-date.
- Actively respond to stakeholder comments.
- Transfer discussion notes, and external information to the issue (as links or discussion/description details). | Product Manager |

| <i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i>Prioritized backlog: The issue and epic backlog is the primary signal stakeholders use to know what's "up next" for a group. The backlog is also the queue for a group to work from, as features progress through the Product Development Flow phases. This queue is kept up to date with milestones and rank ordering on issue boards. | - Regular review of issue prioritization (such as issue board ordering and milestone assignment).
- Align prioritized backlog to category direction and category maturity state.
- Consider using the RICE formula to help make prioritization tradeoffs.
- Consider holding prioritization sessions to collect priority feedback from stable counterparts as there are other DRIs for bug and maintenance work. | Product Manager |

Validation phase 2: Problem validation

<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Label: workflow::problem validation

Key Participants

| Role | Function |

|-----|-----|

| DRI | Product Manager |

| Collaborators | Product Designer
 UX Researcher |

| Informed | Development team
 Other stakeholders as appropriate |

Description

To ensure the right solutions are delivered, the team must start their work with a validated problem. This can take many forms and be achieved through Product Manager, Product Designer and UX Researcher collaboration.

If the problem is documented and well-understood, it may be possible to quickly move through this phase by documenting the known data about the user problem. A documented problem can be categorized as a pre-existing experience from feedback directly from users or an issue that has user engagement confirming that the problem is experienced by multiple users. A well-understood problem can be one that has a series of documented qualitative research from customer interviews led by the Product Manager, triangulating different sensing mechanisms confirming the problem, or using known data. Some examples of known data include Customer Request Issues or pre-existing Actionable Insights from prior research. To document that a problem is well-understood, link the known data and any customer calls to the relevant issues and epics.

If the problem is nuanced or not yet well understood, then it will likely take longer to validate with users properly. This phase's primary outcome is a clear understanding of the problem, along with a simple and clear way to communicate the problem to various stakeholders. Although optional, it is recommended to use an Opportunity Canvas as a tool that helps individuals better understand a problem, and communicate it to various stakeholders. An Opportunity Canvas can also be used to recommend creation of a new category including asking for new resourcing.

Outcomes and Activities

| Outcomes | Activities | DRI |

|-----|-----|-----|

| *Thorough understanding of the problem: The team understands the problem, who it affects, when and why, and how solving the problem maps to business needs and product strategy.* | - Create an issue using the Problem Validation Template in the GitLab project.
- Complete an Opportunity Canvas. - Schedule a review of the opportunity canvas for feedback. - Create an issue using the Problem Validation Research Template in the UX Research project and work with UX Researcher to execute the research study. - Validate your problem with users using any of the proposed methods and document your findings in Dovetail. | Product Manager |

| *Update issue/epic description: A well understood and clearly articulated customer problem is added to the issue, and will lead to successful and efficient design and development phases.* | - Ensure your issue is up-to-date with the latest understanding of the problem. - Understand and document (in the issue) the goals that people want to accomplish using the Jobs to be Done (JTBD) framework. - Conduct continuous interviews with customers on a regular cadence to stay up-to-date on the problems that users face. - Leverage your opportunity canvas to communicate the problem to your stable counterparts and group stakeholders. Consider scheduling a review to gather feedback and communicate the findings to Product and UX leadership. - Confirm that the Product Designer understands the problem well enough to ideate on solutions. | Product Manager |

| Initiate the Dogfooding Process: When validating problems, it's important to gather feedback from internal customers, in addition to the broader community. Capturing internal customer

feedback early on in the product development flow helps ensure their needs are considered as the feature matures, accelerating key Dogfooding outcomes. Driving internal usage of features consistently leads to greater customer adoption and is required for viable maturity. | - PMs are strongly encouraged to open Dogfooding issues during the validation phase and capture internal customer feedback to help inform initial and/or future iterations of a feature. | Product Manager |

Validation phase 3: Design

<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Labels: workflow::design

Key Participants

Role	Function
DRI	Product Designer
Collaborators	Product Manager Engineering Manager Development team UX Researcher Software Engineer in Test Technical Writer Legal & Corporate Affairs
Informed	Product Design Manager Application Security Engineer Other stakeholders as appropriate

Description

After understanding and validating the problem, we can begin or continue to ideate potential solutions through a diverge/converge process. However, if the outcome from the problem validation phase confidently suggests an incremental modification to the existing solution, the aforementioned diverge/converge process could be skipped.

The Product Designer leads the team (Product Manager, Engineering team, UX Researcher, Software Engineers in Test, and Technical Writers, as needed, depending on the item) in ideating potential solutions and exploring different approaches (diverge) before converging on a single solution. Product Managers and the Engineering team evaluate solutions by determining if they meet customer and business goals, are technically feasible, and align with legal compliance considerations. The team is encouraged to engage with stakeholders to determine potential flaws, missed use cases, potential security risks, and if the solution has the intended customer impact.

During this phase the Product Manager and Engineering Manager are both responsible for reviewing the Legal Risk Checklist (accessible to team members only) and determining whether any sections need to be completed. After the team converges on the proposed solution or identifies a small set of options to validate, the issue moves into the Solution Validation phase.

To start the Design phase, the Product Designer or Product Manager applies the workflow::design label to an existing issue or, if needed, creates a new issue with this label.

Outcomes and Activities

Outcomes	Activities	DRI
----------	------------	-----

|-----|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Proposed solution(s) identified and documented: The Product Designer works with the Product Manager and Engineering team to explore solutions and identifies the approach(es) that strike the best balance of user experience, customer value, business value, and development cost. | The Product Designer optionally apply by workflow::ready for design label to the issue, signaling the design backlog of next issues to be done is prioritized.

 Diverge: explore multiple different approaches as a team. Example activities:

- Think Big session.
Internal interviews (be sure to document findings in Dovetail).

 - Creating user flows.

 Converge: identify a small set of options to validate. Example activities:
 - Think Small session with the team.
 - Design reviews with team
 - Low fidelity design ideas.
 - Update issue/epic description with proposed solution. Add Figma design file link or attach design to GitLab's Design Management to communicate the solution idea. Read to understand what tool to use.
 - Validate approach with help from stakeholders. Run user validation using any of the proposed methods and document your findings in Dovetail and appropriate GitLab issue.
 - Draw inspiration from competitive and adjacent offerings. | Product Designer |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Shared understanding in the team of the proposed solution: The Product Designer leads the broader team through a review of the proposed solution(s). | - Review the proposed solution as a team so that everyone has a chance to contribute, ask questions, raise concerns, and suggest alternatives.
- Review the proposed solution with leadership. | Product Designer |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Confidence in the technical feasibility: It's important that Engineering understands the technical feasibility of the solution(s) to avoid rework or significant changes when we start the build phase. | - Discuss the technical implications with Engineering to ensure that what is being proposed is possible within the desired timeframe. When sharing design work, use both Figma's collaboration tools and GitLab's design management features. Read to understand what tool to use.
- Engage engineering peers early and often through Slack messages, pings on issues or by scheduling sessions to discuss the proposal.
- If the solution is large and complex, consider scheduling a spike to mitigate risks and uncover the optimal iteration path. | Product Designer |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Updated issues/epic descriptions: The Product Manager and Product Designer ensure issues and epics are up-to-date. | - Ensure issues and epics are up-to-date, so we can continue our work efficiently and asynchronously.
- Experiment definition. | Product Manager |

|Continue Dogfooding process | - If applicable to their feature, the Product Manager continues the Dogfooding process by deciding whether to build the feature in GitLab or keep outside the product. | Product Manager |

Validation phase 4: Solution Validation

<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Label: <code>workflow::solution validation</code>

Key Participants

| Role | Function |

DR	Product Designer	Product Manager	Development team	Technical Writer	Software Engineer in Test	Application Security Engineer	Other stakeholders as appropriate
Informed							

Description

After identifying one or more potential solutions that meet business requirements and are technically feasible, the Product Manager and Product Designer must ensure that we have confidence that the proposed solution will meet the user's needs and expectations. This confidence can be obtained from work performed during the design phase and supplemented with additional research (including user interviews, usability testing, or solution validation). If necessary, this phase will launch a Solution Validation issue within the GitLab UX Research project which will walk the team through research to validate their proposed solution(s).

In addition, any non-functional requirements for the feature need to be considered and documented. These include such things as evaluating whether or not application limits need to be introduced, or any considerations around data storage should be evaluated. Defining these non-functional requirements up front ensures we are considering scalability and the long-term success of the feature. Sensible default values should be identified at this stage which align with the long-term vision for the feature.

To start the Solution Validation phase, the Product Designer or Product Manager applies the workflow::solution validation label to an existing issue.

Outcomes and Activities

Outcomes	Activities	DR
High confidence in the proposed solution: Confidence that the jobs to be done outlined within the problem statement can be fulfilled by the proposed solution.	Gather feedback from relevant stakeholders.	Product Designer
Documented Solution validation Learnings: The results of the solution validation is communicated to and understood by team members.	Document solution validation findings as insights in Dovetail.	Product Designer

Build track

The build track is where we plan, develop, and deliver value to our users by building MVCs, fixing defects, patching security vulnerabilities, enhancing user experience, and improving performance. This track is also a time when we get insight into whether we are creating the right thing for our users. DRs across engineering disciplines involving Design, Backend, Frontend and Quality work closely together to implement MVCs while in close collaboration with the Product Manager. Decisions are made quickly if challenges arise. We instrument usage and

track product performance, so after MVCs are delivered to customers, feedback is captured quickly for learnings to refine the next iteration. For an example of how to leverage GitLab's various features to create a focused and collaborative board to flow through the Build track, check out this video.

Build Goals & Outcomes

When: As we build MVCs according to our product development timeline

Who: Product Manager, Product Designer, Development team, Software Engineer in Test

What:

- Release to a subset or full set of customers as appropriate.
- Assess UX, functional, technical performance, and customer impact.
- Collect data to measure MVC against success metrics to inform the next iteration.
- Iterate until success metrics are achieved and the product experience is optimal.

Outcome: Deliver performant MVCs that improve one or more of our Product KPIs and/or Engineering KPIs. If it fails to do so, honor our Efficiency value (that includes a low level of shame), abandon it, and restart the validation cycle to identify the right solution.

Build phase 1: Plan

Required Labels

| Label | Usage |

|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::planning breakdown | Applied by the Product Manager on or before the 4th of the month signaling an intent to prioritize the issue for the next milestone. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::ready for development | Issue has been broken down and prioritized by PM for development. Issue also has a work type classification (type::) label and milestone assigned at this point. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Deliverable | Applied to issues by engineering managers indicating it's been accepted into the current milestone. |

Key Participants

| Role | Function |

|-----|-----|

| DRI | Product Manager |

| Collaborators | Product Design Manager
 Engineering Manager
Product Designer
Software Engineer in Test
Development team
Legal & Corporate Affairs|

| Informed | Product Marketing Manager
Customer Success Manager
Technical Writer

 Application Security Engineer|

Description

This phase prepares features so they are ready to be built by engineering. Bugs, technical debt, and other similar changes that are not features may enter the process in this phase (or may benefit from entering in earlier phases based on the cost of doing the work requiring the full problem to be validated to ensure it makes sense to do the work). Following Validation Phase 4 the feature should already be broken down into the quickest change possible to improve the user's outcome and be ready for a more detailed review by engineering. During this phase, Product Managers will surface issues they intend to prioritize for a milestone by applying the workflow::planning breakdown label. At this point, Engineering Managers will assign an engineer to further break down and apply weights to that work. This process is a collaboration between the DRI and Collaborators. Tradeoff decisions can be made and feature issues evolve from validation solutions to clear MVCs that can be delivered in a single milestone. Be sure to document all decisions on issues.

During this phase, the Product Manager and the Engineering Manager must revisit the Legal Risk Checklist (accessible to team members only) to make sure none of their previous determinations during Validation phase 3: Design need revision.

By reviewing and weighing work in the beginning of the Build Track, Product Managers are able to make better prioritization tradeoffs and engineering teams can ensure they've scoped the right amount of work for the milestone. If an issue enters the workflow::planning breakdown state it doesn't necessarily mean it will be prioritized in the next milestone, a Product Manager may make a tradeoff decision depending on capacity, and urgency.

Once work has passed the workflow::planning breakdown step, the workflow::ready for development, type:: labels along with an upcoming milestone are applied to the issue. If an issue has been broken down, but not yet ready to pull into a milestone you may optionally apply the workflow::scheduling label; however, in this state any issue that has the workflow::ready for development label without a milestone has an implied status of "waiting to be scheduled". Engineering Managers and/or Product Design Managers will apply Deliverable label to issues with a milestone and marked workflow::ready for development signaling acceptance of the issue for that milestone. This process occurs at the beginning of milestone planning.

During this phase, it's important to keep Application Security Engineers informed to ensure that they have visibility into planning schedule. This provides them with sufficient time for planning dynamic testing so they can keep the product manager and development team informed of any time/resource requirements.

Outcomes and Activities

| Outcomes | Activities

| DRI |

|-----|

-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Well-scoped MVC issues - Issues are the SSOT for all feature development. |

- Refine issues into something that can be delivered within a single milestone
- Open follow on issues to track work that is de-prioritized
- Promote existing issues to Epics and open implementation issues for the upcoming milestone
- Review feature issues with contributors
- Consider scheduling a POC or engineering investigation issue
- Make scope tradeoffs to reach for a right-sized MVC
- Request an issue review to ensure communication is clear and have proposed the right iteration plan to execute on the solution. | - Product Manager |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Prioritized Milestone - The team should understand what issues should be delivered during the next milestone | - Product Manager sets workflow::ready for development, type::, and a milestone signaling intent to prioritize
 - Product Designer and Engineering Manager apply Deliverable label signaling acceptance of issue in the next milestone - Product Manager creates a planning issue

| - Product Manager, Product Designer, and Engineering Manager |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Defined Quality Plan - Involving SETs in this phase ensures they are able to understand and effectively plan their own capacity before engineering is truly underway.| - Quad Planning
- Test planning

| - Software Engineer in Test |

Implementation Issue Refinement - Some teams have found it useful to treat issue refinement as a separate, iterative task from planning breakdown. This separation allows them to focus backlog refinement on the aspects of the original feature that will be delivered first.| - Further refine implementation issues identified in the workflow::planning breakdown step by additionally applying workflow::refinement.

| - Engineering Manager |

Build phase 2: Develop & Test

Required Labels

| Labels | Usage |

|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::in dev | Applied by the engineer after work (including documentation) has begun on the issue. An MR is typically linked to the issue at this point. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::in review | Applied by an engineer indicating that all MRs required to close an issue are in review. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::blocked | Applied if at any time during development the issue is blocked. For example: technical issue, open question to PM or PD, cross-group dependency. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::verification | After the MRs in the issue have been merged, this label is applied signaling the issue needs to be verified in staging or production. |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-

hidden="true"></i> workflow::awaiting security release | Applied by an engineer after the security issue has passed verification, this label signals that it is ready but awaiting the next monthly security release.|

Key Participants

Role	Function
DRI	Assigned engineer
Collaborators	Product Manager Software Engineer in Test Technical Writer Product Designer Application Security Engineer
Informed	Product Marketing Engineering Manager Cross-stage PM Sales Customer Support

Description

The develop and test phase is where we build the features, address bugs or technical debt and test the solutions before launching them. The PM is directly responsible for overall prioritization of a milestone, including bugs and maintenance work; however, the Engineering Manager and software engineers are responsible for the implementation using the engineering workflow. Engineering owns the definition of done and issues are not moved into the next phase until those requirements are met. Keep in mind that many team members are likely to contribute to a single issue and collaboration is key.

This phase begins after work has been broken down, and prioritized in Phase 1. Work is completed in priority order as set at the beginning of the milestone. The Engineering Manager will assign an issue to an engineer who is responsible for building the feature or addressing a bug or maintenance issue. An engineer can also self-serve and pick up the next priority order issue from the workflow::ready for development queue on their team's board. That engineer will update its workflow:: label to indicate where it's position in the development process.

When an issue is in workflow::in review, the Application Security Engineer would help validate the risk mitigations through the non-blocking application security review process.

Documentation for the work will be developed by the engineer and the Technical Writer (see Documentation with code as a workflow). The Technical Writer should review the documentation as part of the development process. Items discovered during a documentation review should not block issues moving into the next phase. This may drive the creation of follow-on improvement MRs for the documentation, after release.

After the feature code has been merged, the issue should be moved to workflow::verification. When an issue is in workflow::verification, the responsible engineer will manually test the feature in either the Staging or Production environment.

Note: Work deemed out-of-scope or incomplete by engineering is taken back into the plan phase for refinement and rescheduling for completion.

Outcomes and Activities

Outcomes	Activities	DRI
----------	------------	-----

|-----|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Feature is built | - Engineering Manager check that definition of done is met
- Provide regular status updates to stakeholders
- Provide asynchronous updates to avoid status check-ins and synchronous stand-ups
- Engineers follow the engineering process to implement assigned issues. | Engineer |
|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Feature is tested | - Engineers test features they implement (see Definition of done).
- SET sets testing requirements on the issue.
- SET follows up on any specific test coverage changes necessary as an outcome of Quad Planning.
- Technical Writers complete a review of any developed documentation.
- Application Security Engineer validates the risk mitigations through the non-blocking application security review process. | Engineer |

Build phase 3: Launch

Issue Status: Closed

Required Labels

| Label | Usage |

|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> workflow::complete | Applied after the feature is deployed to production, any needed verification is complete, and the issue has been closed. |

Key Participants

| Role | Function |

|-----|-----|

| DRI | Development: Close issue after it's available in production and any needed verification is complete.
Product Manager: Initiate release post item creation if they decide it's warranted.
Product Manager: Continue the dogfooding process if they decide it's applicable.
Product Manager: Consider alerting relevant stakeholders in appropriate Slack channels. |

| Collaborators | Development team, Quality counterpart, and Product Manager may verify the feature is working as expected in production. (Primary verification is, of course, performed prior to production whenever possible.)
-Technical Writers create any documentation issues or MRs required to address issues identified during review that weren't resolved. |

| Informed | Stakeholders for the change (including customers, open-source users, and GitLab team members) will be informed about the feature by the change in the status of the issue or the release post. GitLab team members may also be informed by posts in relevant Slack channels. |

Description

When the change becomes available in production and any needed verification is complete, the issue is closed by the development team and the workflow::complete label is added so stakeholders know work on it has been completed. Afterward, the Product Manager coordinates the release post and dogfooding process when they apply.

Outcomes and Activities

| Outcomes | Activities | DRI |

|-----|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Feature is available to GitLab.com hosted customers: After it's deployed to production (and any feature-flags for it are enabled), the feature is launched and available to GitLab.com hosted customers. | - Code is deployed to production.
- Feature flag(s) enabled. | Development |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Feature is available to self-managed customers: The feature will be available in the next scheduled release for self-managed customers to install. | - Code is included in the self-managed release, (depending upon the cut-off). | Development |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Stakeholders of a feature will know it's available in production | - After the feature is deployed to production and any needed verification in production is completed, the development team will close the issue and add the workflow::complete label.
- Product Manager may follow up with individual stakeholders to let them know the feature is available. | Development |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Customers will be informed about major changes: When appropriate for a change, a release post item will be written and merged by the Product Manager. | - Product Manager follows the instructions in the template, which will then cause it to appear on the GitLab.com releases page and be part of the release post. | Product Manager |

|Continue Dogfooding process | - If the PM wants to Dogfood the feature and it's ready for internal consumption, the Product Manager promotes it internally. | Product Manager |

| Experiment results and follow-up issue is created | For experiments, create a follow-up issue that will be where results of the test and next-steps are tracked. | Product Manager |

Build phase 4: Improve

Label: n/a

Key Participants

| Role | Function |

|-----|-----|

| DRI | Product Manager |

| Collaborators | Product Designer
Customer Success
Sales
Data Analysts
UX Researcher
Product Management Marketer
Technical Writer |

| Informed | Development team
Leadership
Other stakeholders as appropriate |

Description

After launch, the Product Manager and Product Designer should pay close attention to product usage data. This starts by ensuring your AMAU is instrumented and reporting as you expect. From there consider how the feature has impacted GMAU and SMAU. At this point you should also solicit customer feedback to guide follow-on iterative improvements, until success metrics are achieved/exceeded and a decision can be made that the product experience is sufficient. To create a combined and ongoing quantitative and qualitative feedback loop, consideration of the

outcomes and potential activities below are recommended.

Outcomes and Activities

| Outcomes | Activities | DRI |

|-----|-----|-----|

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Understand Qualitative Feedback: To know how to improve something, it's important to understand the qualitative feedback that we're hearing from users and team members. | - Create a dedicated feedback issue (optional).
- Continue dogfooding process.
- Review user feedback in issues.
- Follow up with CSMs and SAEs to gather feedback from interested customers.
- Set up follow-up calls with customers to gather more specific feedback.
- Consider running a Category Maturity Scorecard evaluation.
- Consider running a survey for usability. | Product Manager |

|<i class="fab fa-gitlab fa-fw" style="color:rgb(252,109,38); font-size:1.25em" aria-hidden="true"></i> Measure Quantitative Impact: Qualitative data is great, but coupling it with quantitative data can help to paint the full picture of what is going on. Set up dashboards in Tableau and review the performance and engagement of your change. | - Update any applicable dashboards in Tableau, if necessary work with the data team for more complex reporting.
- Review AMAU, GMAU, and SMAU dash